Gradient Descent: Theory, Variants, and Practical Guidance

A two-column reference with formulas, tables, and visual intuition

Explains the mathematics of gradient descent, compares major variants, and provides practical guidance for using it effectively.

Autor: Maru

February 2026

1. What gradient descent is (in one paragraph)

Gradient descent is an iterative optimization method for minimizing a differentiable objective (or "loss") function. Starting from an initial parameter vector θ_0, it repeatedly moves in the direction that most rapidly decreases the objective—namely the *negative gradient*. Despite its simple update rule, many practical variants exist (batch, stochastic, mini-batch, momentum, Adam, etc.), mainly to improve speed, stability, and robustness on large and noisy datasets.

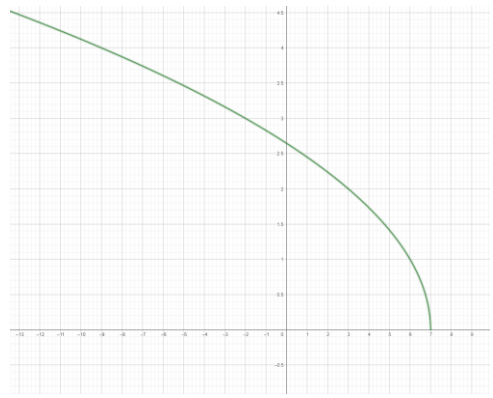
2. Mathematical setup and notation

We seek to minimize a function $J(\theta)$ where $\theta \in \mathbb{R}^d$. In supervised learning with data $\{(x_i, y_i)\}_{i=1}^n$, a common form is an empirical risk: $J(\theta) = (1/n)\sum_i \ell(f(x_i;\theta), y_i) + \lambda R(\theta)$, where ℓ is a per-sample loss, R is a regularizer, and $\lambda \geq 0$ controls regularization strength.

$$\theta^\star \in \backslash\mathrm{argmin}_{\theta\in\mathbb{R}^d} J(\theta)$$

Optimization objective (minimizer may not be unique).

The gradient of J at θ is the vector of partial derivatives: $\nabla J(\theta) = [\partial J/\partial\theta_1, \ldots, \partial J/\partial\theta_d]^T$. It points toward the direction of steepest *increase* of J; therefore $-\nabla J(\theta)$ is the steepest local *decrease* direction.



Function of two variables: the parameter space is often multi-dimensional (θ ▯ ▯^{d}).

3. Derivatives, gradients, and why the gradient matters

In 1D, the derivative $f'(x)$ measures local slope. In multiple dimensions, the gradient generalizes this idea: it is the unique vector whose dot product with a direction u gives the directional derivative. This makes gradients the natural tool for designing local "move rules" that reduce an objective.

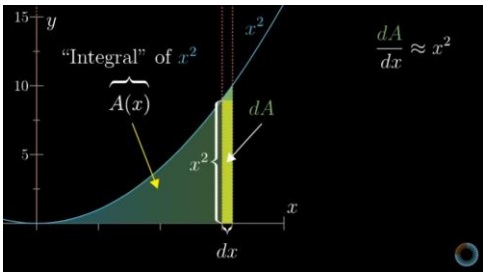
$$D_u J(\theta) = \lim_{\epsilon\to 0} \frac{J(\theta + \epsilon u) - J(\theta)}{\epsilon} = \nabla J(\theta)^\top u$$

Directional derivative equals the gradient dotted with the direction.

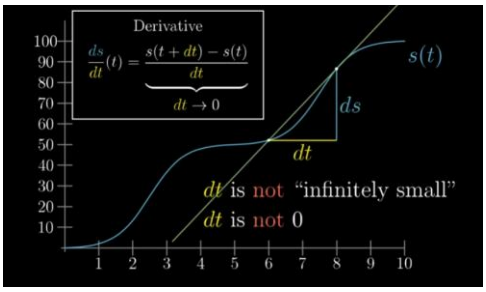
A key justification comes from a first-order Taylor approximation: $J(\theta + \Delta) \approx J(\theta) + \nabla J(\theta)^T\Delta$. If we choose $\Delta = -\alpha\nabla J(\theta)$, then the predicted change is approximately $-\alpha\|\nabla J(\theta)\|^2$, which is non-positive when $\alpha>0$.

$$J(\theta + \Delta) \approx J(\theta) + \nabla J(\theta)^\top \Delta$$

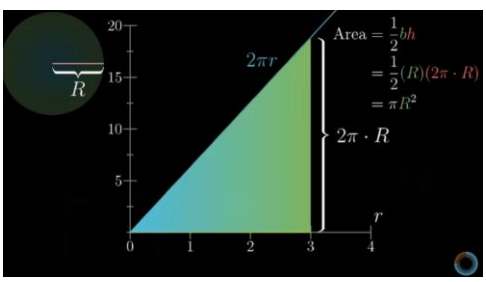
First-order (linear) approximation that motivates gradient descent.



Derivative as a limiting slope: intuition behind "follow the slope downhill."



Derivative and finite differences: in computation, gradients are approximated or computed via backprop/automatic differentiation.



Area/accumulation picture: optimization often integrates many small improvements over iterations.

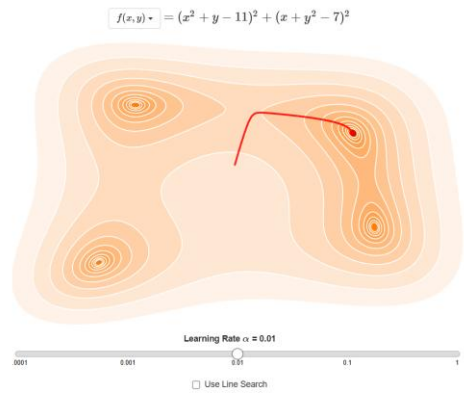
4. The core gradient descent algorithm

At iteration t, compute the gradient g_t = ∇J(θ_t) and update parameters by stepping against it. The scalar α>0 is the learning rate (step size). Choosing α well is essential: too large causes divergence/oscillation; too small makes progress slow.

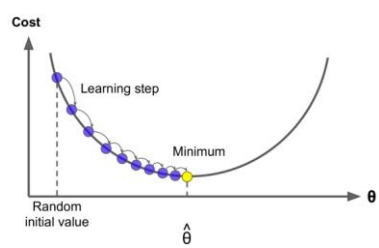
$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

Basic gradient descent update rule.

1. Initialize θ₀ (randomly or from a heuristic).
2. Repeat until a stopping condition is met: compute gradient g_t and update θ.
3. Stop when ‖g_t‖ is small, validation loss stops improving, or after a fixed budget of steps.



Typical 1D descent trajectory: repeated steps move toward a minimum of the cost curve.



Gradient vector and its components: the update subtracts a scaled gradient from the current point.

5. Types of gradient descent: batch, stochastic, and mini-batch

The main distinction is how the gradient is estimated from data. Using all n samples gives a low-noise but expensive step. Using one sample is cheap but noisy. Mini-batches interpolate between

these extremes and are the standard choice in modern deep learning.

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta \ell_i(\theta) (\text{with } \ell_i(\theta) = \ell(f(x_i;\theta), y_i))$$

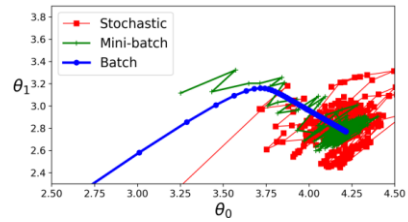
Full (batch) gradient as the average of per-sample gradients.

$$g_t = \frac{1}{|B_t|}\sum_{i \in B_t} \nabla_\theta \ell_i(\theta_t)$$

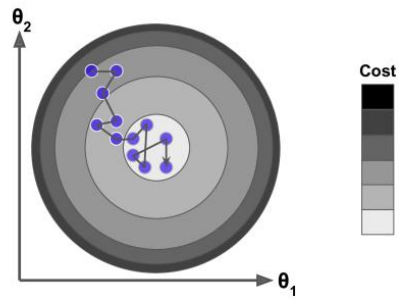
Mini-batch gradient estimate using a batch B_t of samples.

Comparison of the three core gradient descent regimes.

Variant	Gradient estimate	Pros	Cons	Typical use
Batch GD	All samples	Stable descent; deterministic for fixed data	Slow per step; expensive on large n	Small/medium datasets; convex problems
Stochastic GD (SGD)	Single sample	Very fast updates; can escape shallow local structure	Noisy path; needs scheduling/averaging	Large-scale learning; online learning
Mini-batch GD	Small batch (e.g., 32–409	Efficient on GPUs; reduced noise; good generalization	Still stochastic; batch size tuning	Default for deep nets



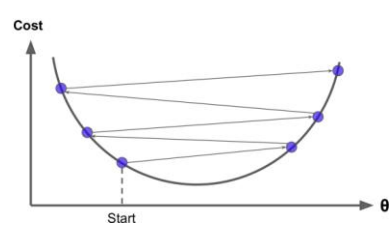
Batch vs mini-batch vs stochastic trajectories: noise level changes the path and convergence behavior.



Contour plot intuition: SGD produces a jittery path while still trending toward lower cost regions.

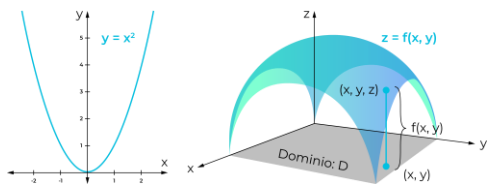
6. Learning rate (step size): stability, speed, and schedules

The learning rate α controls how far each step moves. If α is too large, updates may overshoot minima or even diverge. If α is too small, training can stall. In practice, α is often scheduled over time (decayed) or adapted per-parameter (e.g., Adam).



Overshooting due to a large learning rate: steps jump across the valley instead of settling into it.

6) lization



Learning-rate sensitivity: the step size strongly shapes the descent path and convergence speed.

Common learning-rate strategies and when they are useful.

Schedule	Example formula (α_t)	When it helps	Notes
Constant	α	Convex/smooth problems with tuned α	Simple but brittle if curvature varies
Step decay	α·γ^{⌊t/k⌋}	When progress plateaus	Common in deep learning training recipes
Exponential decay	α·exp(−kt)	Smooth long runs	May decay too fast if k is large
Cosine decay	α_min + (α_max−α_min)·(1+cos(πt/T))/2	Modern deep learning; stable late training	Often combined with warmup
Warm up	α_t increases for first W steps	Large-batch training stability	Reduces early divergence
Line search (classic optimization)	Choose α_t to satisfy sufficient decrease	Well-behaved smooth objectives	More expensive per step; less used in deep nets

7. Convergence basics (what theory says)

Convergence guarantees depend on assumptions. For convex and *L-smooth* functions (gradient is Lipschitz), a step size α ≤ 1/L ensures the objective decreases monotonically. For *μ-strongly convex* functions, gradient descent converges linearly to the unique minimizer.

$$\|\nabla J(\theta) - \nabla J(\phi)\| \le L\|\theta - \phi\|$$

L-smoothness (Lipschitz gradient) assumption.

$$J(\theta_{t+1}) \le J(\theta_t) - \frac{\alpha}{2}\|\nabla J(\theta_t)\|^2 \text{for } \alpha \le \frac{1}{L}$$

A common descent inequality for smooth objectives.

$$\|\theta_t - \theta^\star\| \le (1-\alpha\mu)^t\|\theta_0 - \theta^\star\| \text{for } 0 < \alpha \le \frac{1}{L}$$

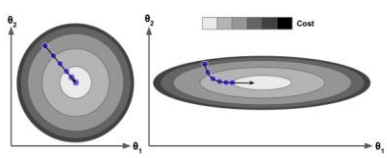
Linear convergence under μ-strong convexity (one typical form).

In non-convex problems (typical for deep neural networks), the best general guarantee is often convergence to a point with small gradient norm, i.e., ‖∇J(θ_t)‖ → 0 in some sense. Practical success

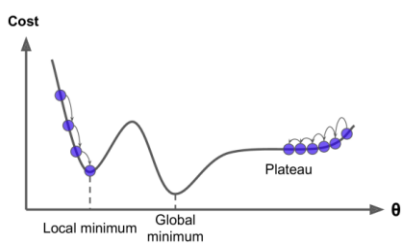
comes from good regularization, architectures, and robust optimizers.

8. Geometry and pitfalls: ravines, plateaus, and local structure

Real objectives may have very different curvature along different directions (ill-conditioning). This creates "ravines": steep walls but a shallow floor. Standard gradient descent then zig-zags. Plateaus (small gradients) slow progress, and saddle points can trap methods that rely purely on first-order information.



Contours with different axis scaling: anisotropic curvature produces curved or zig-zagging trajectories.



Local minima vs global minima and plateaus: non-convex landscapes can slow or mislead descent.

Feature scaling and normalization are crucial because they change the conditioning of the problem. In linear models, standardizing features often turns a narrow ravine into a more spherical bowl, allowing larger stable learning rates and faster convergence.

9. Momentum and accelerated first-order methods

Momentum accumulates a running "velocity" vector, smoothing noisy gradients and reducing zig-zagging in ravines. It can be interpreted as a low-pass filter on gradients and is often one of the most effective upgrades over vanilla SGD.

$$v_{t+1} = \beta v_t + \nabla J(\theta_t), \theta_{t+1} = \theta_t - \alpha v_{t+1}$$

Gradient descent with (classical) momentum, with β ∈ [0,1).

Nesterov accelerated gradient (NAG) evaluates the gradient at a look-ahead point, which can improve stability and theoretically improves rates for certain convex problems.

$$v_{t+1} = \beta v_t + \nabla J(\theta_t - \alpha\beta v_t), \theta_{t+1} = \theta_t - \alpha v_{t+1}$$

One common form of Nesterov momentum (look-ahead gradient).

10. Adaptive learning-rate methods (AdaGrad, RMSProp, Adam)

Adaptive methods scale updates per parameter based on historical gradient magnitudes. They are especially helpful when different parameters have different natural scales or when gradients are sparse. However, their generalization behavior can differ from SGD+momentum, so selection is task-dependent.

$$r_t = r_{t-1} + g_t \odot g_t, \theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{r_t} + \epsilon}$$

AdaGrad (elementwise), with g_t a mini-batch gradient estimate.

$$r_t = \rho r_{t-1} + (1-\rho)(g_t \odot g_t), \theta_{t+1} = \theta_t - \alpha \frac{g_t}{\sqrt{r_t} + \epsilon}$$

RMSProp: exponential moving average of squared gradients.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, v_t = \beta_2 v_{t-1} + (1-\beta_2)(g_t \odot g_t)$$

Adam maintains first and second moment estimates.

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

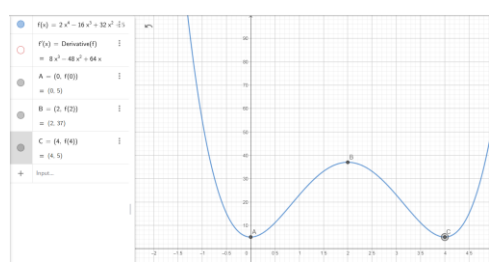
Adam bias correction and parameter update.

Common first-order optimizers related to gradient descent.

Optimizer	Key idea	Typical hyperparameters (common defaults)	Good for	Cautions
SGD	Noisy first-order steps	α tuned; optional schedule	Strong generalization; simple baselines	Needs careful α schedule
SGD + Momentum	Velocity smooths gradients	β≈0.9, α tuned	Ravines; faster convergence	Can overshoot if α too large
Nesterov	Look-ahead gradient	β≈0.9	Stability + speed in some settings	More sensitive to implementation details
AdaGrad	Per-parameter scaling (accumulates)	α, ε≈1e−8	Sparse gradients	Learning rate can decay too much (e.g., NLP features)
RMSProp	EMA of squared gradients	ρ≈0.9, α, ε	Non-stationary objectives	Hyperparameter coupling
Adam	EMA of first+second moments	β_1=0.9, β_2=0.999, ε=1e−8	Fast training; robust defaults	May need weight decay variant (AdamW)

11. Stopping criteria and diagnostics

Practical stopping is usually based on validation performance (early stopping). In pure optimization, you may also stop when the gradient norm is small, the relative improvement in J is below a tolerance, or after a maximum iteration budget. Tracking learning curves helps detect divergence, overfitting, or stalled progress.



First-derivative criterion intuition: at an optimum, the derivative/gradient is (approximately) zero.

- **Divergence**: loss becomes NaN/inf or increases rapidly → reduce α, add gradient clipping, check data scaling.

- **Oscillation**: loss bounces without improving → reduce α or increase momentum carefully; consider scheduling.
- **Slow progress**: near-constant loss → increase α slightly, improve conditioning (normalization), or use momentum/Adam.
- **Overfitting**: training loss decreases while validation worsens → add regularization or stop earlier.

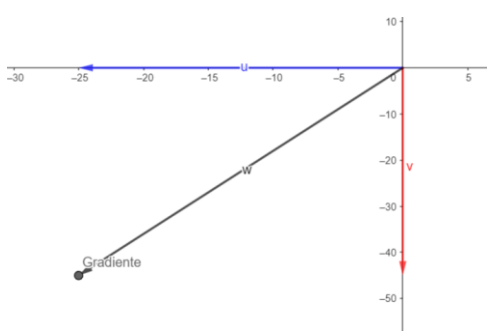
12. Worked examples (short)

Example A (1D quadratic). Let $J(\theta) = (1/2)a\theta^2$ with $a>0$. Then $\nabla J(\theta)=a\theta$ and $\theta_{t+1}=(1-\alpha a)\theta_t$. This converges to 0 if $|1-\alpha a|<1$, i.e., $0<\alpha<2/a$. The larger a (curvature), the smaller the stable step size.

$$J(\theta) = \frac{1}{2}a\theta^2 \;\Rightarrow\; \theta_{t+1} = (1-\alpha a)\theta_t$$

Closed-form recursion for a simple quadratic objective.

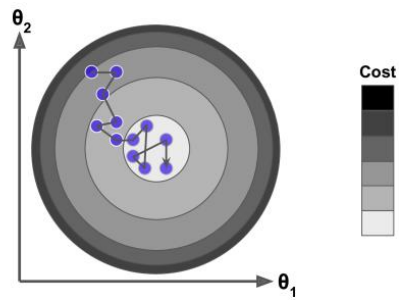
Example B (non-convex contour landscape). Even with a fixed α, gradient descent can be pulled toward different basins of attraction depending on initialization. In higher dimensions, saddle points and narrow valleys become more common, motivating momentum and adaptive methods.



Gradient direction in 2D: the update decomposes into coordinate-wise components of the gradient.

[Image Placeholder: (If present) Learning dynamics can depend heavily on α and initialization; visualization aids understanding.]

(If present) Learning dynamics can depend heavily on α and initialization; visualization aids understanding.



Cost contours with iterative steps: descent tends to follow gradients perpendicular to level sets.

13. Practical checklist (applied gradient descent)

4. **Scale inputs**: standardize features or use normalization layers; improves conditioning.
5. **Pick a baseline**: start with SGD+momentum or Adam depending on domain norms.
6. **Tune learning rate first**: it usually dominates performance; then tune batch size and regularization.
7. **Use schedules**: decay α and consider warmup for large batches.
8. **Monitor**: track training/validation loss and gradient norms; add gradient clipping if needed.
9. **Reproducibility**: set seeds and log hyperparameters for fair comparison.

14. Summary

Gradient descent minimizes differentiable objectives by stepping in the direction of the negative gradient. Batch, stochastic, and mini-batch variants trade off compute cost against gradient noise. Learning rate strategy is critical, and geometry (conditioning) strongly influences behavior. Momentum and adaptive optimizers (RMSProp/Adam) often improve robustness, but good results still depend on careful monitoring and tuning.Suggested next topics: second-order methods (Newton, quasi-Newton), constrained optimization, proximal methods, and the theory of stochastic approximation for SGD.